WillowBrook Ranch Report

Source Code Google Drive Link:
https://drive.google.com/file/d/1TKthFJ590KUWdwhkNeBt48CVRoXyBYaj/view?usp=sharing

1. Game Description:

WillowBrook Ranch is an open-world farming simulator where the player can freely explore three connected areas: the Farm, the Town, and the Beach. Unlike our midterm project, which was level-based with task requirements, this game focuses on player choice and interaction. The player can plant and water crops, fish at the docks through a reaction-based minigame, shop at the town markets, and eventually interact with NPCs and local markets as the town continues to grow. Our goal for WillowBrook Ranch is to create a cozy, small-town environment that feels handmade and lived-in with mechanics that gradually expand as the player progresses.

The game begins with the player waking up on their ranch and stepping into a world that uses a full day/night cycle, scene-swap transitions, and responsive character animations. As development continues, the player will gain access to new crop types, a working market system for buying and selling goods, inventory management, and more advanced fishing and NPC features. WillowBrook Ranch is designed to be simple, charming, and open-ended, giving players the freedom to farm, explore, and build their own routine within the town.

2. Gameplay Description:

1. **Starting Screen:**
   The first thing the player sees when launching WillowBrook Ranch is the Starting Screen. This page captures the tone of the game: bright, inviting, and peaceful. It includes three main options: Start Game, Credits, and Exit. Selecting Credits opens a pop-up window that lists each developer and their GitHub profile, displayed clearly against the same illustrated background as the main menu. The player is able to close this popup window by pressing the “X” icon in the upper right corner of the popup window.



Figure 1 - Credits Screen

If the player wants to exit the game from the starting screen, they can click the “Exit” button which runs a script that stops the application. Meanwhile, if the user wants to start the game, they can click the “Start Game” option.

This feature keeps transparency in development and gives the game a polished, completed feel. The simplicity of the Starting Screen ensures the player is not overwhelmed when launching the game from their Unity Hub. It’s also a place to keep the credits and extra information if we needed to provide anything else before the player could start the game.



Figure 2 - Starting Scene

2. **Farm:**

The Farm is the heart of WillowBrook Ranch and the first major area the player explores. It contains a neatly arranged field of 16 crop plots, a small farm house for the player to skip to the next day at, a well, a broken windmill that can be further developed into having another purpose, and a few trees to surround the land. The player spawns into the world with 50g, 5 tomato seeds, 5 carrot seeds, 5 mushroom seeds, and 5 eggplant seeds. They're able to use these seeds at their garden and can use their gold to buy fish from the fisherman's market at the beach or used to buy more seeds from the town's main market.



Figure 3 - Farm Spawn

If you go up to the crop plots, you'll be able to plant mushrooms, carrots, eggplants, or tomatoes Every crop has a different buy cost and sell value, allowing players to experiment with profit strategies as they farm. Players can plant (pressing "p"), water (pressing "space"), and harvest (pressing "h") crops using the toolbar at the bottom of the screen.

If the player wants to skip to the next day, they can go back to their house. Interacting with the door (which uses a trigger event that tracks if the player is in the house's collider), opens a pop-up asking whether the player wants to skip to the next day, which advances the game's day/night cycle and refreshes their routines. To cancel out of this, the player could walk away from the popup window to cancel.



Figure 4 - Skip To Next Day Popup

Each day, the player starts at 8:00am. If the player doesn't make it back to their house by midnight, then they'll receive another popup window notifying them that they've passed out. Pressing the "confirm" button will allow the player to respawn in front of their house (regardless of the scene they were in) and reset the day for 8:00am.

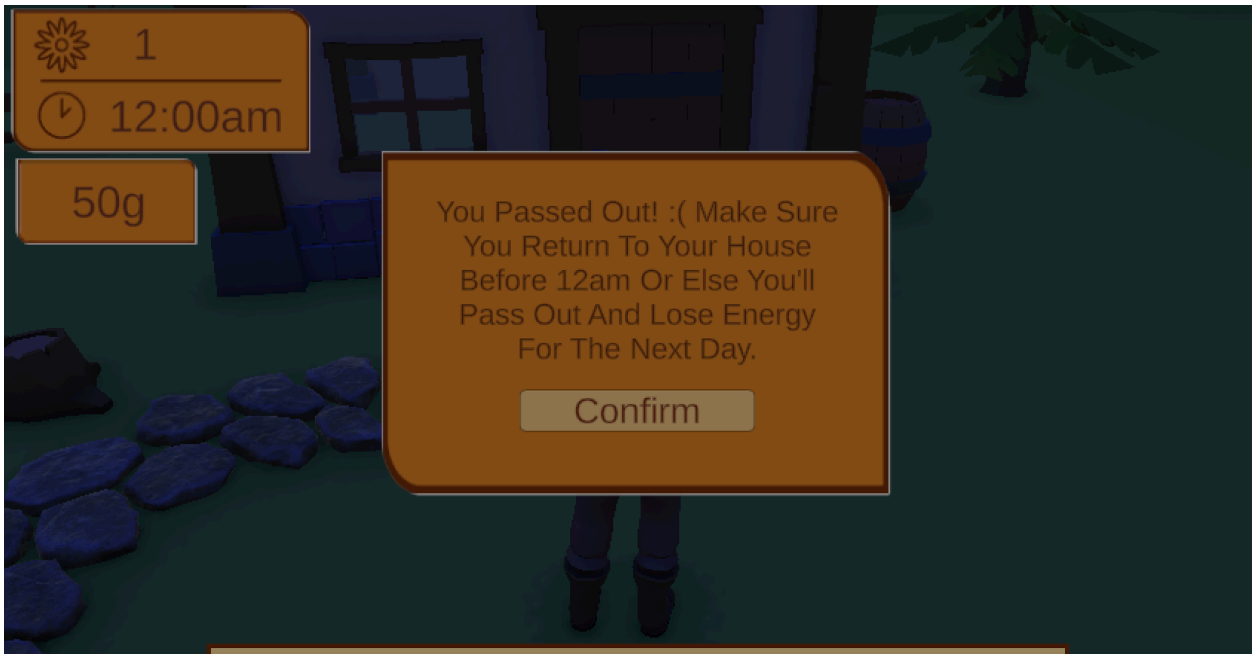


Figure 5 - Passed Out Popup

The Farm was built entirely by hand. Every terrain mound, fence, rock path, and garden plot was individually placed to create an organized yet charming layout. This is where the

main farming mechanics begin and it sets the tone for the slow-paced, rewarding gameplay loop.

3. **Town:**
   The Town acts as the game's social and economic hub. It includes two NPCs the player can talk to, with dialogue windows that form the foundation of future friendship and quest systems. Even though the friendship tracker was the single feature not completed this semester, the NPC interactions serve as the stepping stone for those future mechanics.



Figure 6 - Town



Figure 7 - NPC 1



Figure 8 - NPC 2

The Town also features a fully functional market system. The UI is clear and straight-foward, walking up to one of the buildings (the "General Market Store building") pops up a window showing all seed prices and all sell values, letting the player make their farming decisions. It follows the same logic as the house's popup window for if the player wanted to skip to the next day.



Figure 9 - General Market

Each seed option in the General Store can be used on the player's farm land to grow a profit from. They can also sell their crops at this market for a higher profit amount

compared to the seeds. If they wish to close out of this window, they can either walk away or press the “X” button in the upper right corner.

Like the Farm, the Town was completely built from scratch. Each building, pathway, and piece of scenery was placed intentionally to give the town a “small but growing” feel as though the player’s success could eventually help expand it. This area showcases the game’s NPC system, economy system, and early worldbuilding.

4. **Beach:**
   The Beach introduces a completely different type of gameplay: fishing. Players can walk up to the docks and cast their line, triggering a randomized fishing minigame. The game selects either an easy or hard fish, requiring fast key presses and quick reactions to successfully catch it.

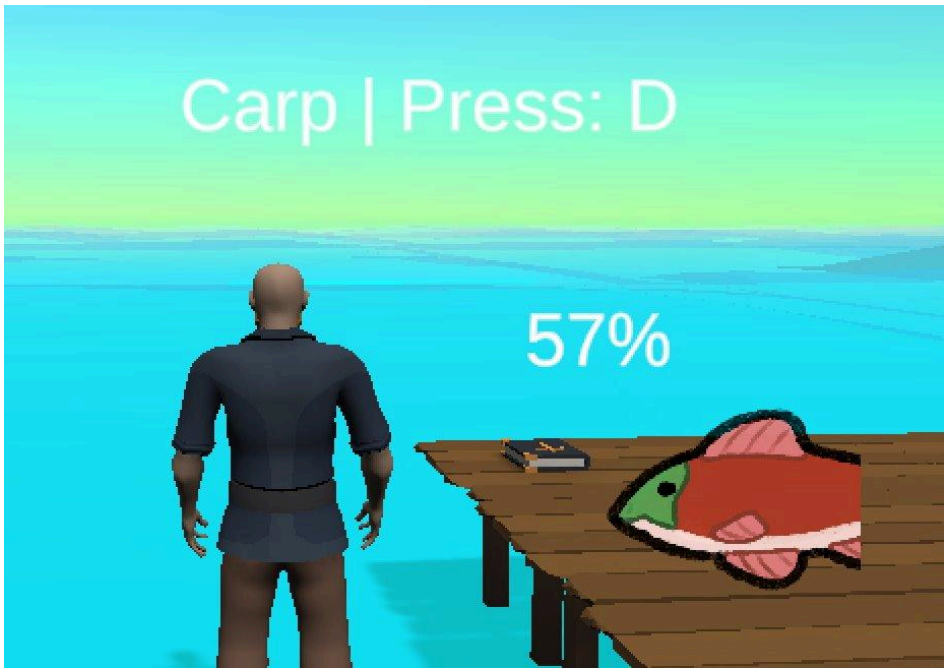


Figure 10 - Fishing Minigame

Once caught (or lost), the player can walk over to the fish market, where they can sell their fish for gold. This gives players an alternative income source outside of farming. The player is also able to buy fish at the same market (for a higher price than what they could sell for) in case they wanted extra fish for a future quest or for energy.



Figure 11 - Fish Market

The Beach's bright colors and open layout create a calm atmosphere that contrasts with the farmland and town scenery. It is designed to be a “breathing space” where players can step away from planting tasks and enjoy a different style of gameplay.



Figure 12 - Beach

3. System Design:

When we were designing our game, we wanted to make sure our code would be organized and clean so each of us wouldn't be too confused when needing to rely on each other's code. We created the 4 main scenes to be separate (Starting Scene, Farm, Town, and Beach) because we

didn't want anything that may affect one area to affect another in case of errors or bugs. We also knew that our worlds/scenes would be smaller compared to other games purposely in order to make sure it stayed with a "small town" aesthetic. If we were to code our game to be larger, we would've needed to break WillowBrook into small segments so the game wouldn't lag while playing it.

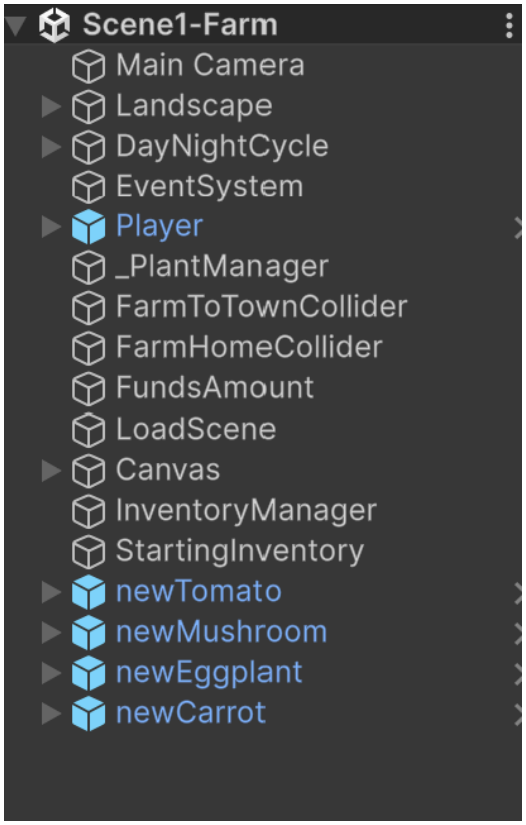


Figure 13 - Farm Hierarchy



Figure 14 - Landscape

For how we tried to organize our hierarchy, we often made objects children of others. For example, when we were creating the landscape in each Scene, we created an empty gameobject called “Landscape” followed by creating children of that gameobject named after different objects we’d use to decorate and create our scene with. Below those, we added our tiles, rocks, buildings, etc.

In order to make sure everything was organized and easily found, we tried to make sure everything was in their proper folders and were named with comments. For example, we kept the majority of the scripts in a Scripts folder, our downloadable assets in the Downloadable Assets folder, our Scenes in the Scenes folder, with the rest of the organization following the same format.



Figure 15 - Assets

For anything that would remain to be 3D in our game, we used prefabs that we downloaded from the Unity Asset Store. This was because it was the most familiar to us and with coding compared to the earlier projects we used in class. Anything that was a popup or something that could be 2D, we used canvases. Using both 3D and 2D helped us keep things organized and not affect one another too much in order to keep our game simple but still complex.

4. System Implementation:

We organized our scripts by system so it was easier to navigate the project as it grew. For example, player-related logic (movement, interactions) was kept in player scripts, while farming, inventory, fishing, and UI each had their own sets of scripts. Most scripts are attached directly to GameObjects in the scene (such as the Player, crop plots, shop doors, or UI Canvas), and we relied on serialized fields in the Inspector to connect references like UI panels, sprites, and object prefabs.

We tried to follow clear naming conventions so it would be easier to read and debug our code later. Script names and variables are descriptive of their purpose. This helped us quickly understand where certain functionality lived, especially when switching between systems like farming, markets, and fishing.



Figure 16 - Scripts Folder

We implemented player movement using a script attached to the player character that reads input from both WASD and the arrow keys. The script updates the player's position every frame and allows the player to move in multiple directions, including diagonals, giving smoother control over exploration. We also added sprinting mechanics so that holding the shift button lets the player move faster across the map.

The camera is set to follow the player and rotate with them instead of being static. In the Unity editor, the camera has a script for when the player moves, the camera smoothly follows and keeps them centered.

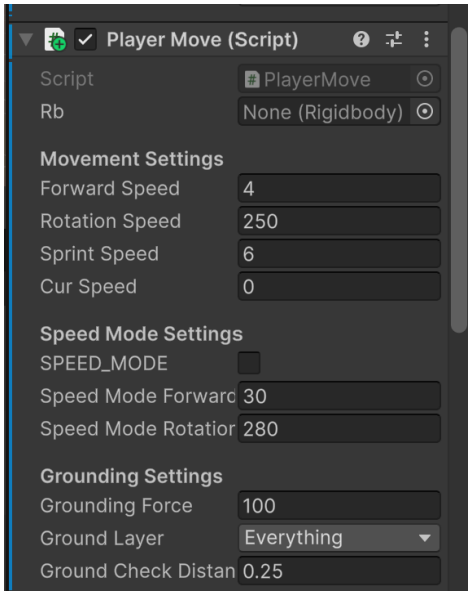


Figure 17 - Player Movement

```
void FixedUpdate()
{
    // update curSpeed based on the sprinting state and speed mode
    if (SPEED_MODE)
    {
        curSpeed = speedModeForwardSpeed;
    }
    else if (IsSprinting)
    {
        curSpeed = sprintSpeed;
    }
    else
    {
        curSpeed = forwardSpeed;
    }

    MoveAndRotatePlayer();
    ApplyGroundingForce();
}
```

Figure 18 - Player Movement 2

Our farming system is implemented through a combination of tile-based crop plots and scripts that track the state of each plot throughout the planting, watering, and harvesting process. Each of the 16 plots on the Farm has its own script that determines whether it is empty, planted, growing, or ready to be harvested.

The player interacts with plots using specific keys:

- P: Plant a seed
- Spacebar: Water the planted crop

- H: Harvest the crop once fully grown

When the player presses one of these keys near a plot, the plot script checks its current state and performs the correct action. For example, pressing “P” on an empty plot plants the selected crop type, pressing “Space” on a planted plot updates the plot to “watered” and pressing “H” on a fully grown plant harvests it and adds the crop to the player's inventory. We used similar logic to our midterm project where the player needs to wait a few days and continuously water the plants in order to harvest them.

The markets in the Town and at the Beach use a combination of trigger colliders and UI scripts. The player has a script attached to them where if they walk into something with a specific tag (in this case, a Market tag), then it triggers a OnTrigger Event, If it turns true, then it enables a Canvas panel with buttons for buying and selling items.

```
private void OnTriggerEnter(Collider other)
{
    if (other.CompareTag("GeneralStoreTag"))
    {
        if (GUIUpdater.Instance != null)
        {
            GUIUpdater.Instance.LoadGeneralStoreGUI();
        }
        else
        {
            Debug.LogWarning("GUIUpdater.Instance is null - make sure a GUIUpdater is in the scene.");
        }
    }

    if (other.CompareTag("FishMarketTag"))
    {
        if (GUIUpdater.Instance != null)
        {
            GUIUpdater.Instance.LoadFishMarketGUI();
        }
        else
        {
            Debug.LogWarning("GUIUpdater.Instance is null - make sure a GUIUpdater is in the scene.");
        }
    }
}
```

Figure 19 - Market Tags

The UI elements are set up in Unity’s Canvas system, and the buttons are linked to functions in our market scripts that modify the player’s inventory and funds.

```
// FUNCTION TO BUY TOMATO SEEDS
public void BuyTomatoSeeds()
{
    Debug.Log("Attempting to buy Tomato Seeds for " + tomatoSeedPrice + " gold.");

    if(FundsAmount.Instance.playerMoney >= tomatoSeedPrice) // Check If The Player Has Enough Gold
    {
        FundsAmount.Instance.playerMoney -= tomatoSeedPrice; // Deduct The Price
        InventoryManager.Instance.AddItem(tomatoSeedItem, 1); // Add to inventory
        Debug.Log("Tomato Seeds Purchased. Remaining Gold: " + FundsAmount.Instance.playerMoney); // Successful purchase
    }
    else
    {
        Debug.Log("Not enough gold to buy Tomato Seeds."); // Insufficient funds
    }
```

Figure 20 - Buy Tomato Seeds Function

The inventory system keeps track of how many crops or fish the player currently has. When the player buys seeds from the Town market, their funds are reduced and seed counts are increased. When they sell crops or fish, the correct amount of gold is added to their funds and the items are removed. These values are updated in code and displayed on screen so the player always has feedback on their current money and items.

```
    //get the slot for the current item being held (assumes hotbar index 0)
    public InventorySlot GetCurrentHeldItem(){ return GetSlot(0); }

    public bool AddItem(ItemData item, int amount = 1)
    {
        if (item == null || amount <= 0) return false;

        // 1) Try stacking into existing stacks
        for (int i = 0; i < slots.Count; i++)
        {
            var slot = slots[i];
            if (!slot.IsEmpty && slot.item == item && slot.amount < item.maxStack)
            {
                int space = item.maxStack - slot.amount;
                int toAdd = Mathf.Min(space, amount);
                slot.amount += toAdd;
                amount -= toAdd;

                if (amount <= 0)
                {
                    NotifyChanged();
                    return true;
                }
            }
        }
```

Figure 21 - Inventory Script

The fishing minigame is implemented as a separate interaction that activates when the player walks up to the left side of the dock on the Beach and presses the “F” key. A trigger collider near

the dock detects the player and enables the fishing script. It randomizes a timer which is supposed to indicate you “casted your line” and when the timer goes out, you press “Space” to continue. Following the randomizer, it chooses either an easy or hard fish for the player to catch, and then starts a sequence where the player must press certain keys quickly.

We used timers and input checks in the script to determine whether the player pressed the keys fast enough. If you also press the wrong key, you also lose extra time in your timer.

```
public void StartMinigame()
{
    if (player != null && !player.HasEnoughEnergy(energyCostToStart))
    {
        if (keyPromptText != null)
            keyPromptText.text = "You are too tired to fish!";
        return;
    }

    if (player != null)
        player.UseEnergy(energyCostToStart);

    ChooseRandomFish();

    if (playerMove != null)
        playerMove.enabled = false;

    waitingForReplay = false;
    currentFill = startFill;
    active = true;

    NewKey();
    UpdateUI();
}
```

Figure 22 - Fish Start Minigame

Our animation system is managed through a Unity Animator state machine that controls how the player transitions between actions such as walking, harvesting, watering, picking fruit, talking, and idling. The diagram shows a centralized structure where Idle acts as the default state, and all other animations can transition to or from it based on player input and interaction events. For example, pressing movement keys triggers a transition from Idle to Walk, while performing farming actions triggers transitions into Harvest, Watering, or PickingFruit. The Talking state is also included to support NPC interactions. Each transition is driven by parameters updated in our scripts, allowing gameplay logic and animation flow to stay in sync. This setup keeps our

animation handling clean, ensuring that the player always responds visually to whatever action they are performing in the world.

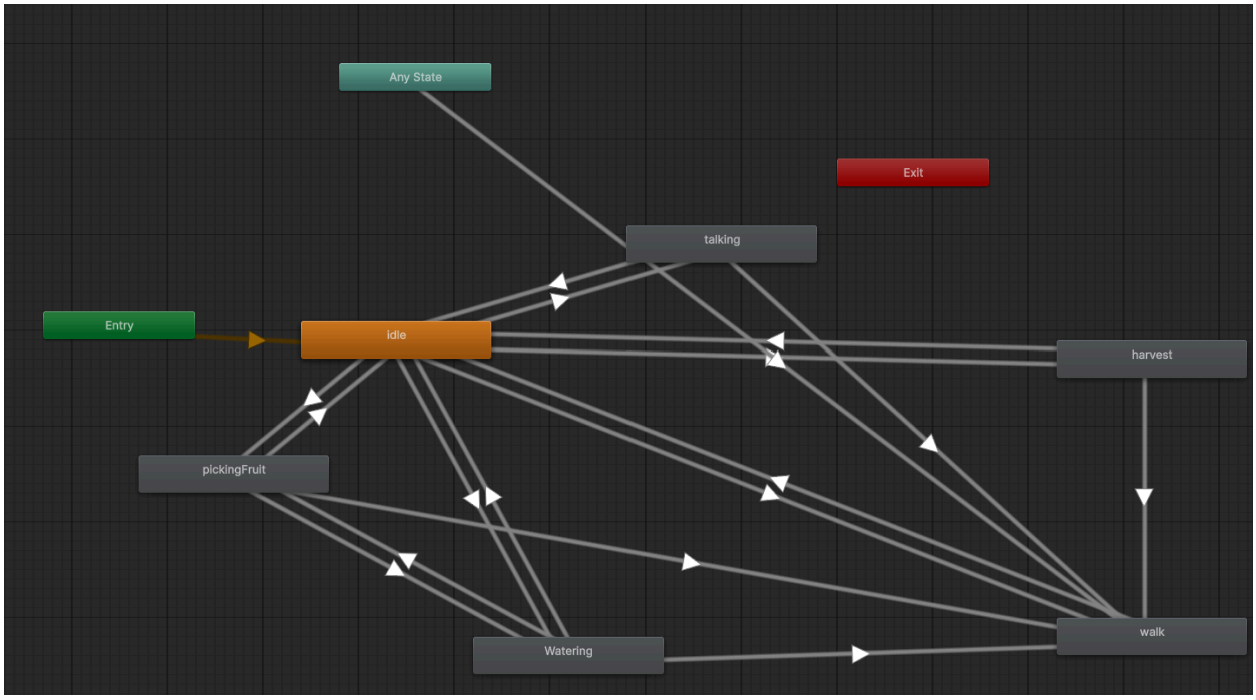


Figure 23 - Animation Handling

Overall, we feel that the quality of our code is solid for a first full game project, especially considering that none of us had used Unity or built a game before this class. We used meaningful naming conventions, separated logic into different scripts instead of placing everything into one file, and tried to keep each script focused on a clear responsibility. If we had more time, we would have gone back to better understand each other's code, which would have helped us as the project became more complex. Doing this earlier would have made integrating our work much easier and likely would have saved us time when combining different systems toward the end of development.

5. Asset References:

All assets below weren't modified! We kept them exactly how they were.

2-D GUIs:

1. **Stardew Valley Tomato Seeds, Carrot Seeds, Eggplant Seeds, Fall Seeds, Largemouth Bass and Carp**
    a. We grabbed these from the official Stardew Valley wiki! Our GUI artist wasn't able to draw us seed packets so we used some from Stardew Valley.

2. **Custom GUIs For Clock, Day, Gold, And Popup Windows**
    a. These were commissioned by one of our friends! We're asking her to create an online portfolio that we can credit her with, but currently we've contacted her via her number and discord.

3D Assets:

1. **Low Poly Farm Pack Lite:** https://assetstore.unity.com/packages/3d/environments/industrial/low-poly-farm-pack-lite-188100

2. **Wood Set Pieces:** https://assetstore.unity.com/packages/3d/props/wood-set-pieces-33853

3. **Lite Farm Pack:** https://assetstore.unity.com/packages/3d/environments/industrial/lite-farm-pack-low-poly-3d-art-by-gridness-243315

4. **Low Poly Fantasy Medieval Village:** https://assetstore.unity.com/packages/3d/environments/fantasy/low-poly-fantasy-medieval-village-163701

5. **Low Poly Medieval Characters Lite:** https://assetstore.unity.com/packages/3d/characters/humanoids/low-poly-medieval-characters-lite-316247

6. **Plants Pack Vegetable:** https://assetstore.unity.com/packages/3d/props/plants-pack-vegetable-304007

7. **Stylized Tomato:** https://assetstore.unity.com/packages/3d/props/free-pack-stylized-tomato-323039

8. **Low Poly Simple Nature Pack:** https://assetstore.unity.com/packages/3d/environments/landscapes/low-poly-simple-nature-pack-162153

9. **Stone Pathways:** https://assetstore.unity.com/packages/3d/props/exterior/modular-stone-paths-284417

10. **Low Poly Medieval Characters** https://assetstore.unity.com/packages/3d/characters/humanoids/low-poly-medieval-characters-316246

6. Reflection:

None of us had ever used Unity before this class, and for all of us, this was the first time we had ever attempted to build a complete game from scratch (and in general). Because of that, the entire semester felt like stepping into a completely new world of development. Learning how Unity worked was a major challenge at first. But over time, these concepts became familiar, and it was exciting to see how each homework assignment slowly introduced the tools and skills we would later rely on for our final project.

One of the most rewarding parts of this course was realizing how much everything connected. Concepts that seemed small or confusing in homework such as writing scripts that respond to player input, creating UI pop-ups, or using event triggers made perfect sense when we applied them to the midterm. Building our first farming simulator showed us what we were capable of, and it motivated us to push even further with WillowBrook Ranch by expanding every system we learned: adding more complex farming mechanics, introducing fishing, designing functional markets, and building three fully custom scenes. Instead of creating a level-based game, we wanted to apply everything into one open-world game so we could test how each thing would affect one another properly.

Even though the class was challenging, especially since we started with zero experience, it was also genuinely fun. Working together as a team, learning through trial and error, and watching the game become more polished each week made the experience feel creative and collaborative instead of overwhelming. Looking back, the process taught us not only how to use Unity and C#, but also how to communicate, divide tasks, solve unexpected problems, and think like actual game developers. This project gave us confidence we didn't have at the start of the semester, and it showed us that even with no prior experience, we were capable of building something we're proud of.

7. Self-Directed Learning:

Throughout the development of WillowBrook Ranch, each of us encountered features and tools that required us to take what we learned in class and expand it into new contexts and ideas. For example, we built simple shops during the midterm, but for this project we wanted them to feel more interactive and integrated into the world. We combined what we already knew with new techniques, such as using TriggerEvents, so that whenever the player approached a shop door, the collider would activate a pop-up window on the Canvas with the market buttons. This approach helped us connect Unity's UI system with world interactions in a way we hadn't tried before.

Another example of self-directed learning was how we handled scene transitions. We used TriggerEvents and colliders to allow the player to walk naturally between the Farm, Town, and

Beach without manually selecting scenes from a menu. Even though WillowBrook Ranch is another farming simulator, our goal was to continue pushing ourselves beyond the techniques we used in the midterm. By experimenting, researching, and teaching ourselves new methods, we strengthened our Unity knowledge and built a project that reflects real growth in our personal development as game creators.

8. Proposal vs Final Product:

Most of everything we wanted to add into our project from the proposal we successfully added! The only things that we are missing or could expand with are friendship meters, quests, an energy bar, and making our inventory more accessible with switching between what the player is holding (possibly with a mouse).

Looking back, a reason for this could've been our project organization and underestimating how much of the later scripts and game objects would reference one another. The markets, the inventory, the fishing minigame and the garden plots all correspond and connect with one another which held us back when we'd struggle with fixing bugs that related to one of those tasks. Another issue could've been because of our workload in other classes, so we didn't take into account how busy our later weeks into the semester would be compared to when we first got this project around Week 10.

What We Added From The Proposal	**What We Were Missing From The Proposal**
Day/Night Cycle	Energy Bar
3 Main Scenes	Quests
Fishing Minigame	Friendship Meter
Inventory & Hotbar	Moving Around Our Inventory Better To Access Different Items
Audio	
2 NPCs	
2 Different Markets	
4 Crop Types	
WASD Walking Mechanics	
Animations	

Passed Out & Skip To Next Day Popups	

9. Team Collaboration & Trello Board:

1. Role Split Table

Role Split Table			
	Roles	**List of duties in detail**	**Summary of the duties**
Kelly Frear	Project Manager & Coder	Manage Github repository Created Trello Create project foundation Storyboarding Manage core mechanics Level design Testing Debugging Keeps track of code base Find assets Created the Farm and Town scene Created the FarmFunds Created The Day/Night Cycle And Day Counter	Was a huge part in actually getting the game up and running. Led the creation of the farm scene, the town scene, and got together all the possible assets of the game.Was the one who contacted an artist to get art for our game. She also designed the levels with the assets, making the game feel like a real farming simulator. Created many of the key features of the game, including; the day/night cycle, the character funds, the markets, and the sleep function. Led report production as well.

Diego Torres	Project Coordinator & Coder	Level design Documentation Testing Debugging Manage Find assets Character Mechanics Manage gameplay Created the crop mechanics Work on animations	Was responsible for creating the farming functionality. All of the farming tasks were created by him, including the different plants, the different stages of plants, etc. Also helped connect the inventory system to the actual farming system, so that the game flows correctly.
Daniel Pangilinan	Coder	Level design Fishing mini game NPC interactions Debugging Testing	Helped design the levels and was responsible for creating the fishing mini game we see on the beach scene. Helped throughout the whole process of the project, even on other tasks, to help with bug fixing and implementation of functions. Also created the NPC system with interaction for the player. Each NPC can have their own dialogue and model, and the character can go speak with them.
Benjamine Medina	Sound engineer, Documentation & Coder	Manage sound design Find assets for sound to be used in the game Created Beach Scene	Designed the initial inventory system with all of its layout, and designed the beach scene. Was also responsible for all the sounds in the game. Created the item system, helped with level design, and bug fixing.

		Be a main contributor to the documentation of our project Manage timeline and distribution of documentation work Will manage documentation In charge of sound engineering	
Owen Man	Coder (GUI Focused)	In charge of GUI aspects Handled The Inventory UIs Made Sure Everything Connected With The Inventory Debugging Testing	Helped finish a big aspect of the inventory system. Finalized the items going and entering the player's inventory. Helped with debugging and testing the inventory, and other functions of the game.

2. Contribution File:

Teamwork contribution form

To evaluate individual contribution to the team, please complete the below form, list out your contribution to the HW/project and % of contribution in regard to the total outcome**. Individual grade will be adjusted based on the contributions.**

Also, please include 2 short paragraphs or bullet points with reflections as a group on what worked well and what needs improvements for the next group assignment work time.

For example:

If all 4 people contributed equally, you can write down 25% for everyone. If 1 member is outstanding and spent a lot of time on the HW/project or solved a challenging question, you can give that person maybe 40% with others being 20%. The contribution percentage is for reference only that helps you to rethink how to better work in a team. If you worked hard for the HW/project, you will have a list of items that you can write about in the contribution items list, like "fixed … bug", "found out how to use … function", "programmed … script that does ….". Your contribution percentage can be told via the items you listed out as well.

Full Name	List of items that you contributed to the HW/Project (bullet points)	% of contribution make by you
Kelly Frear	• Created Both Markets & Their Original Buttons • Bug Tested • Made Sure The FarmFunds Passed Over To Each Scenes • Worked On Documentations • Created The Powerpoint	20%

Diego Torres	• Updated the farming functionality, and now there are multiple plants in the game • Worked on developing the inventory system to work with the farming system • Added the four different crop types • Worked on powerpoint • Modified and resized the NPC dialogue	20%
Owen Man	• Added the seeds & fish as game objects • Buying/selling work with price/inventory/restrictions • Removed all placeholder/test code • Create StartingInventory script/object in Scene1 for anything you want player to start with • Had to remake hotbar bc it went missing from scene 1 • Moved inventory down a bit • Inventory now hides hotbar (bc it duplicate storage) • Added a speed toggle on player object • Made it to where fishing adds fish to inventory	20%

Daniel Pangilinan	• Updated and finished the fishing game with a new fish UI • Created NPC's with dialogue, and the UI that goes with them • Made the easier and harder version of the fishing minigame	20%
Benjamine Medina	• Created the initial inventory system • Added sounds to all aspects of the game • Fixed bugs with npc UI • Worked On Documentation • Worked On Powerpoint	20%

In this HW/project assignment, what went well for your group work?

Answer (1 short paragraph or a few bullet points):

- Handling different tasks as a group really worked well for us. We were all able to take on different roles, and complete different things, and then push our changes to the main of the github, and keep working well off of each other.

- Communicating, and having people designated to certain tasks kept us focused on little goals to achieve our main goal of completing the game

In this HW/project assignment, what problems did you encounter working as a team, any possible solutions to try out for next time?

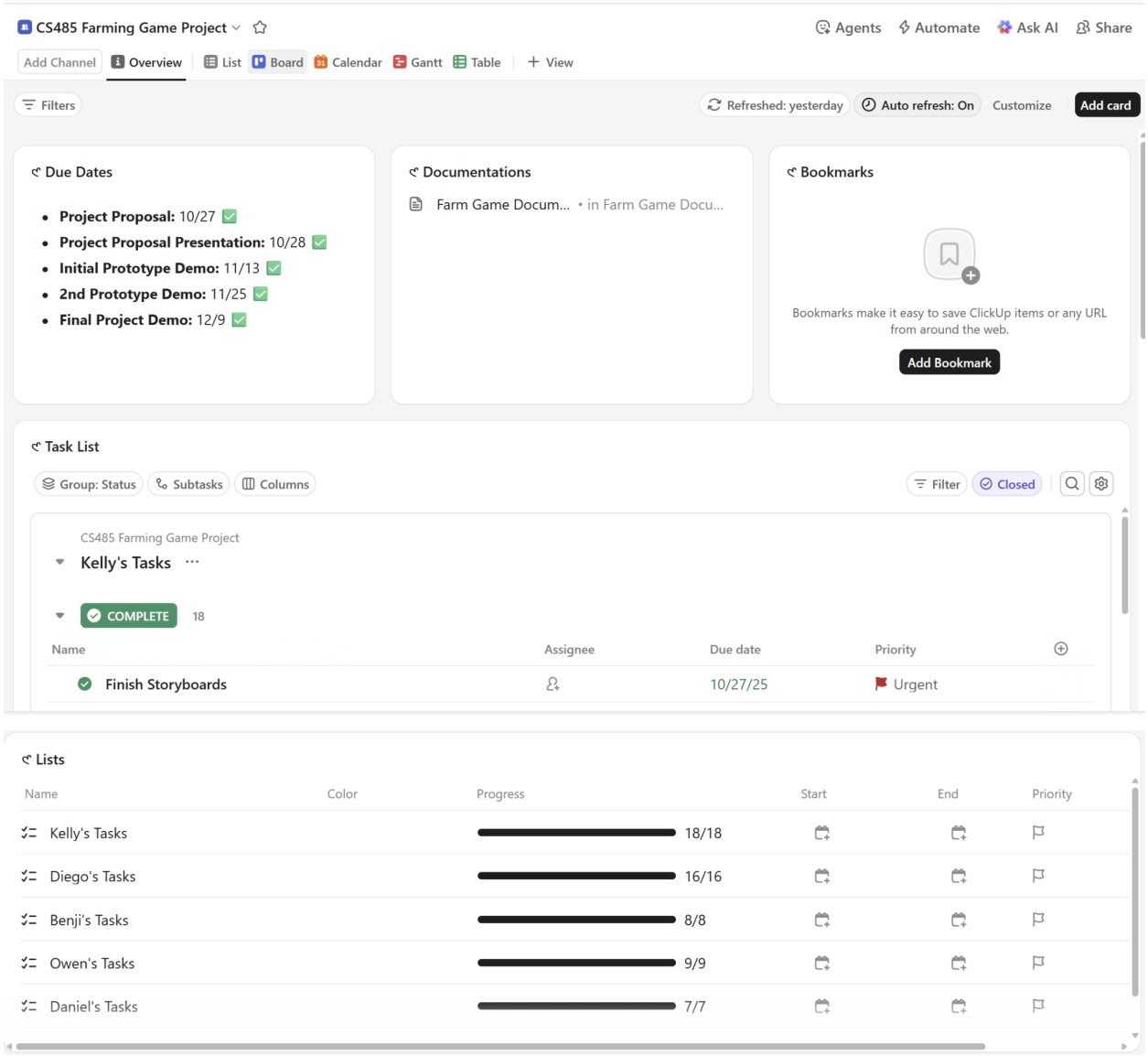
Answer (1 short paragraph or a few bullet points):

- The main problem we encountered this Sprint was our time management. Since it was nearing the end of the semester, we were more focused with other finals and finishing other projects that were due sooner, which took away our focus on this project. Eventually, we all came together and were able to finish our tasks and our project.

3. Trello:

To keep our tasks organized, we used Trello to keep track of our tasks and deadlines. We would update any tasks that were in progress or what needed to be done to make sure we stayed on track.

Our Trello Link: https://app.clickup.com/9017354455/v/s/90171808969


CS485 Farming Game Project
Agents
Automate
Ask AI
Share
Add Channel
Overview
List
Board
Calendar
Gantt
Table
View
Filters
Refreshed: yesterday
Auto refresh: On
Customize
Add card
Due Dates
Project Proposal: 10/27
Project Proposal Presentation: 10/28
Initial Prototype Demo: 11/13
2nd Prototype Demo: 11/25
Final Project Demo: 12/9
Documentations
Farm Game Docum... • in Farm Game Docu...
Bookmarks
Bookmarks make it easy to save ClickUp items or any URL from around the web.
Add Bookmark
Task List
Group: Status
Subtasks
Columns
Filter
Closed
CS485 Farming Game Project
Kelly's Tasks
COMPLETE
18
Name
Assignee
Due date
Priority
Finish Storyboards
10/27/25
Urgent
Lists
Name
Color
Progress
Start
End
Priority
Kelly's Tasks
18/18
Diego's Tasks
16/16
Benji's Tasks
8/8
Owen's Tasks
9/9
Daniel's Tasks
7/7